ATM Transaction Sequence Simulation

Problem statement

Process deposits and withdrawals, maintain a simulated account balance, enforce a withdrawal limit, reject invalid transactions, and produce a transaction summary.

Assumptions and rules

• The opening balance cannot be negative.

• Transactions are tuples: (type, amount). Type is DEPOSIT or WITHDRAW.

• Amounts must be positive.

• A withdrawal is accepted only if it does not exceed the per-transaction limit or available balance.

Algorithm

1. Set the opening balance and withdrawal limit.

2. Read or define the transaction list.

3. For each transaction, unpack its type and amount.

4. Reject non-positive amounts and unknown transaction types.

5. For a deposit, add the amount to the balance and deposit total.

6. For a withdrawal, check the transaction limit and available balance before subtracting.

7. Count accepted and rejected transactions.

8. Display closing balance and totals.

Pseudocode

```
START\nINPUT opening_balance, withdrawal_limit, transactions\nbalance = opening_balance\nFOR each transaction\n  UNPACK typ
```

Flowchart

```
[Start] -> [Set opening balance and limit]\n   |\n[Read next transaction]\n   |\n{More transactions?} --No--> [Print summar
```

Python solution

The program uses beginner-level Python constructs from the assigned syllabus: variables, data types, operators, conditions, loops, strings, lists, tuples, and unpacking where appropriate.

```
opening_balance = 5000
withdrawal_limit = 3000
transactions = [
    ("DEPOSIT", 1500),
    ("WITHDRAW", 2000),
    ("WITHDRAW", 4000),
    ("WITHDRAW", 6000),
    ("DEPOSIT", 500),
    ("WITHDRAW", -100),
    ("TRANSFER", 200)
]

balance = opening_balance
deposit_total = 0
withdrawal_total = 0
accepted = 0
rejected = 0

for transaction in transactions:
    transaction_type, amount = transaction

    if amount <= 0:
        print("Rejected:", transaction, "- amount must be positive")
```

```
        rejected = rejected + 1
    elif transaction_type == "DEPOSIT":
        balance = balance + amount
        deposit_total = deposit_total + amount
        accepted = accepted + 1
        print("Accepted deposit:", amount)
    elif transaction_type == "WITHDRAW":
        if amount > withdrawal_limit:
            print("Rejected:", transaction, "- limit exceeded")
            rejected = rejected + 1
        elif amount > balance:
            print("Rejected:", transaction, "- insufficient balance")
            rejected = rejected + 1
        else:
            balance = balance - amount
            withdrawal_total = withdrawal_total + amount
            accepted = accepted + 1
            print("Accepted withdrawal:", amount)
    else:
        print("Rejected:", transaction, "- unknown transaction")
        rejected = rejected + 1

print("Opening balance:", opening_balance)
print("Deposits:", deposit_total)
print("Withdrawals:", withdrawal_total)
print("Closing balance:", balance)
print("Accepted:", accepted, "Rejected:", rejected)
```

Sample output

```
Accepted deposit: 1500\nAccepted withdrawal: 2000\nRejected: ('WITHDRAW', 4000) - limit exceeded\nRejected: ('WITHDRAW', 60
```

Explanation / observations

This is an educational simulation only. It does not connect to a bank or perform real transactions.

Conclusion

This solution models the stated case using a clear sequence of input, processing, decisions, repetition, and output. The assumptions and sample data can be changed to test other cases.